

Hybrid Music Harvest & Curation Engine

Comprehensive Architecture Blueprint & Analytical Framework v3.0

I. Macro Architecture Overview

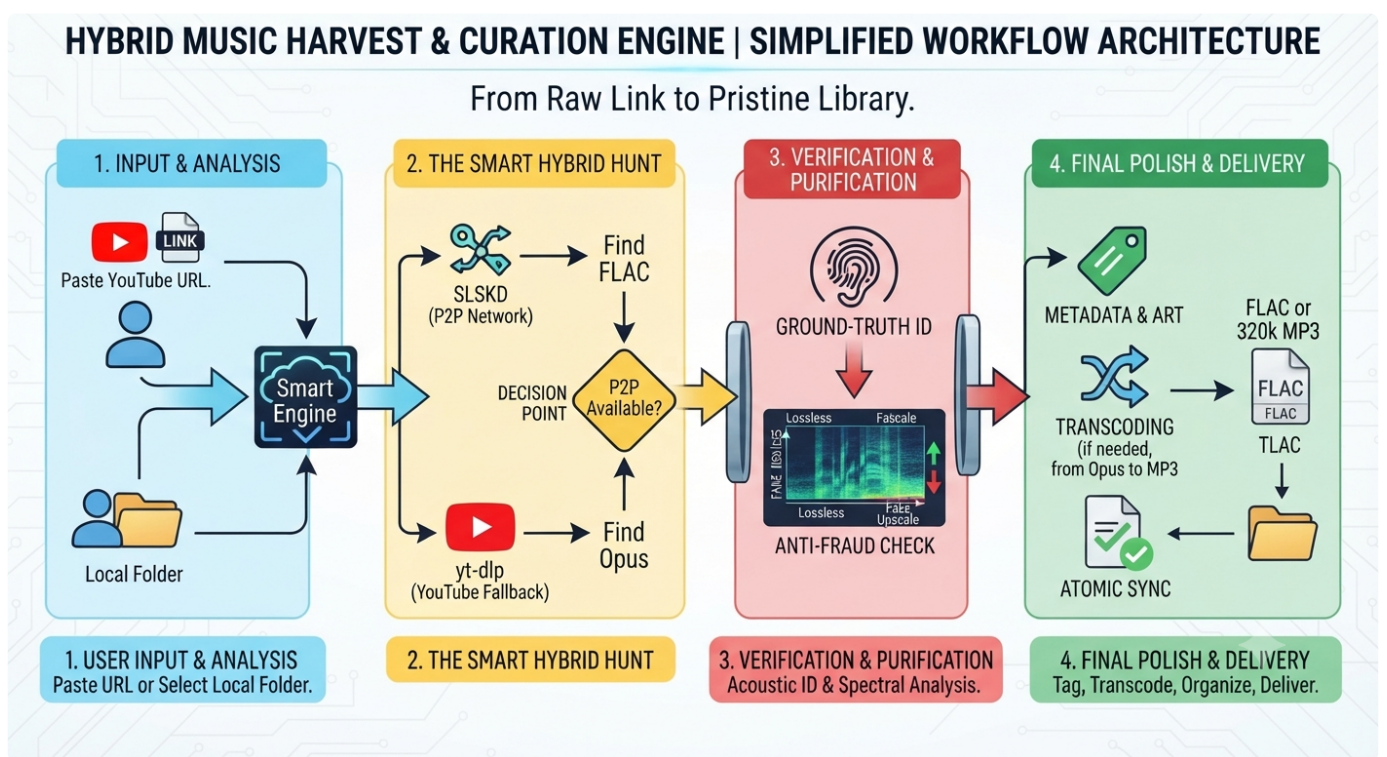


Figure 1: End-to-end data flow illustrating the journey from unverified input vectors to a cryptographically checked, locally synchronized audio library.

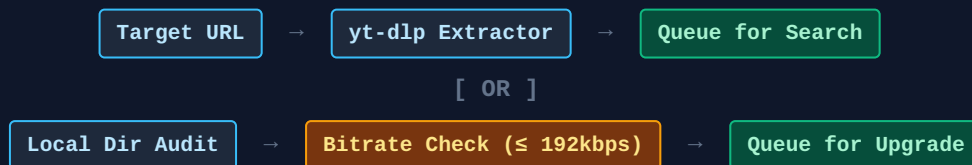
II. Phase-by-Phase Technical Pipelines

Phase 1

Ingestion & Heuristic Filtering

To address both ad-hoc listening and long-term library curation, the architecture provides two primary ingestion vectors. Rather than assuming all local files are flawed or all web links yield poor audio, this phase applies a heuristic filter to establish a baseline quality metric before committing computational resources to the rest of the pipeline.

GRAPH 1.0: INGESTION ROUTING LOGIC

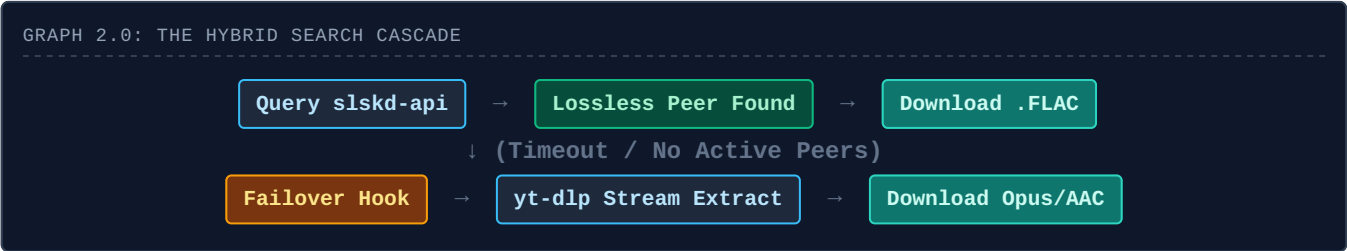


- **Vector A (Targeted Harvesting):** Users input standard URLs (e.g., YouTube). The system utilizes yt-dlp strictly as a lightweight metadata parser to strip away the video container and isolate the track title and channel data, preparing it for the P2P search.
- **Vector B (Library Remediation):** Users point the system to existing local directories. A recursive scan utilizing Mutagen or FFprobe acts as a quality gate. Files demonstrating a bitrate at or above 320kbps are bypassed to save processing overhead, while legacy files (often 128kbps relics) are pushed into the active queue.

Phase 2

Cascading Acquisition Strategy (slskd + yt-dlp Fallback)

Peer-to-peer (P2P) networks are unparalleled for archiving true lossless audio but suffer from inherent asynchronous latency and offline peers. To guarantee a 100% fulfillment rate, this phase employs a cascading acquisition model.

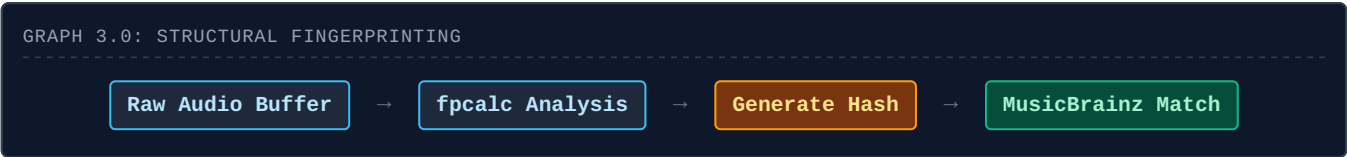


- **Primary Vector (P2P Network):** The backend leverages a localized Souseek daemon (slskd). The REST API is queried specifically for .flac or .alac extensions. If an active peer with an open queue is found, the true lossless file is pulled.
- **Failsafe Vector (Stream Ripping):** If the P2P request hits a timeout threshold, the system gracefully degrades. It invokes yt-dlp to capture the platform's native, highest-fidelity audio stream (typically Opus or AAC format). This ensures the user is never left empty-handed.

Phase 3

Acoustic Ground-Truth Verification

A critical flaw in both P2P sharing and YouTube uploads is "metadata entropy"—files are frequently mistitled, misattributed, or littered with spam tags (e.g., "track01_final_rip_real.mp3"). We circumvent textual metadata entirely by analyzing the structural audio waveform.

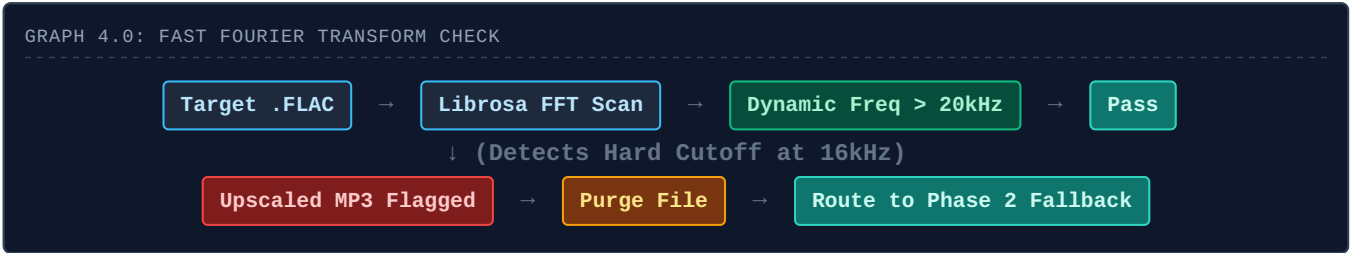


- **Waveform Analysis:** Using Chromaprint (via the fpcalc binary), the system scans the first 30 seconds of the track to generate a unique digital acoustic fingerprint, treating the audio as a structural signature.
- **Database Resolution:** This hash is checked against the AcoustID and MusicBrainz databases via pyacoustid. The result is immutable, canonical metadata: the exact ISRC code, official artist, release year, and proper album title.

Phase 4

Spectral Integrity Analysis (The FFT Gate)

A pervasive issue in lossless music curation is "transcode fraud," where malicious or ignorant users upscale low-quality 128kbps MP3s and save them into FLAC containers. To the naked eye, it looks lossless; to the ear, it sounds compressed. We solve this mathematically.

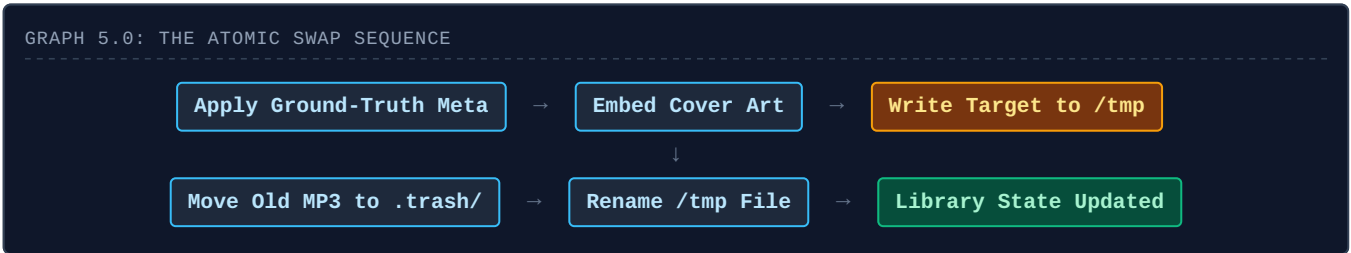


- **Frequency Domain Mapping:** The system utilizes Python's `librosa` or an FFmpeg spectral scan to perform a Fast Fourier Transform (FFT). True lossless files maintain rich frequency data extending well beyond 20 kHz.
- **Cutoff Detection:** Lossy MP3 compression notoriously applies a strict low-pass filter, creating a "brick wall" cutoff at roughly 16 kHz. If the FFT detects this artificial shelf inside a FLAC container, it flags the file as fraudulent, deletes it, and reroutes the track to the YouTube Opus fallback.

Phase 5

Asset Standardization & Atomic File Operations

The final phase is responsible for data hygiene and safe integration. Audio files must be uniformly tagged and safely swapped into the local filesystem without risking the corruption of the user's existing collection.



- **Standardization:** The canonical data retrieved in Phase 3 is permanently embedded into the track's ID3 or Vorbis comments, alongside high-resolution album art fetched via the MusicBrainz API.
- **Transcoding Protocol:** If the track was sourced from the Opus fallback, FFmpeg transcodes it into a universally compatible 320kbps CBR MP3. Verified P2P FLACs are left in their native, untouched state.
- **Atomic Synchronization:** To prevent data loss (e.g., if the app crashes mid-write), the system performs an atomic swap. The new file is written to a temporary path. The old legacy file is then safely moved to an archive directory, and the new file instantly assumes its original filename and location.